Session 4.2

Krusell–Smith, Solved Three Ways

Generalised moments, a histogram, a history of shocks: three ways to put a moving distribution into a network, and what each can and cannot see

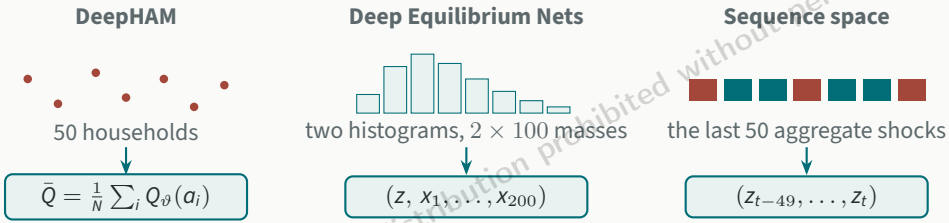


© Zhigang Feng
Redistribution prohibited without permission

Zhigang Feng

The pages marked PRELIMINARY are based on preliminary research by Zhigang Feng, Jiequn Han, Thomas J. Sargent, and Shenghao Zhu (September 2026). Do not circulate without the authors' permission.

Three Ways to Give a Network a Moving Distribution



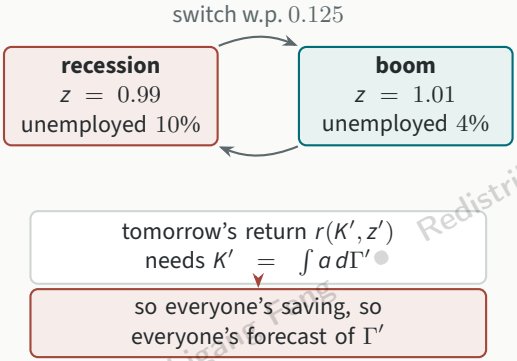
Redistribution prohibited without permission

© Zixiang Feng

- Krusell and Smith's economy: the household's state is $(a, \varepsilon; z, \Gamma)$, and Γ is a distribution.
- Each replaces Γ by something finite. For each: what the network reads, what it minimises, *where its training distributions come from*.
- **Then Deck 4.1's method, which writes down how Γ moves.**

Source: Han, Yang & E, *QE* (2026); Azinovic, Gaegauf & Scheidegger, *IER* (2022); Azinovic-Yang & Žemlička (2025).

The Economy, and Why the Household Must Forecast Γ

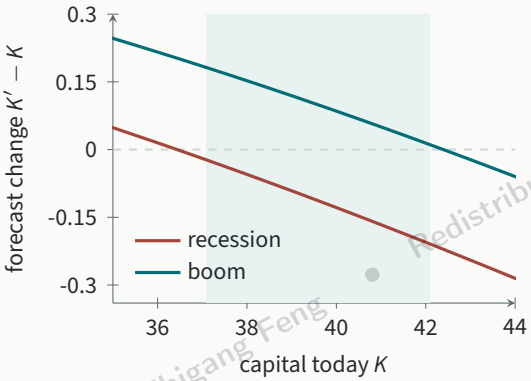


- $V(a, \varepsilon; z, \Gamma) = \max_{a' \geq 0} \ln c + \beta \, \mathbb{E} \, V(a', \varepsilon'; z', \Gamma')$, $c + a' = (1 - \delta + r)a + y(\varepsilon)$.
- r, w are marginal products at $K = \int a \, d\Gamma$; employment $\varepsilon \in \{0, 1\}$ moves with z.
- $\Gamma' = H(\Gamma, z, z')$**: the law of motion every household needs and nobody is given.**

Source: Krusell & Smith, *JPE* (1998); Den Haan, Judd & Juillard, *JEDC* (2010).

© Zhigang Feng
Redistribution Prohibited without permission

The Classical Answer: Forecast With the Mean



- The bet: $\log K' = \kappa_0(z) + \kappa_1(z) \log K$. The state shrinks to (a, ε, K, z).
- Solve at the rule, push a histogram $3{,}000$ quarters, regress, update: 17 CPU-seconds.
- $R^2 > 0.99999$ one step ahead. Run alone for $2{,}000$ quarters: mean miss 0.054%, worst 0.187%.
- **The benchmark, and the idea each deep method generalises.**

© Zhigang Feng
Redistribution prohibited without permission

Source: Krusell & Smith (1998); Den Haan, *JEDC* (2010); the lab, block B. Quarterly, as published: $\alpha = 0.36$, $\beta = 0.99$, $\delta = 0.025$, $u = \ln c$, $z \in \{0.99, 1.01\}$.

Four Questions Every Deep Method Answers

	DeepHAM	Deep Equilibrium Nets	Sequence space
1. what stands for Γ	learned moments of a 50-household panel	a 100-point histogram per employment state	the last 50 aggregate shocks
2. what is minimised	minus lifetime utility over a rollout; a value regression	Euler and KKT residuals	a Fischer–Burmeister Euler residual
3. where training states come from	simulated panels, refreshed	16 simulated economies, carried forward	a replay buffer of simulated histories
4. what the gradient passes through	150 simulated periods of the panel	one period, at stored next states	one period; prices held fixed

- **Row 3: in all three, training distributions come from simulating the network's own policy.**

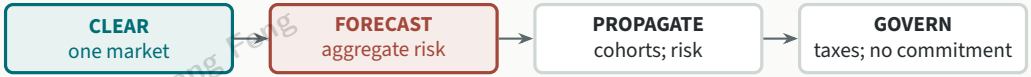
Source: DeepHAM's public repository; the Deep Equilibrium Nets continuum-of-agents code; the sequence-space tutorial.

© Zhigang Feng Redistribution prohibited without permission

Session 4.2

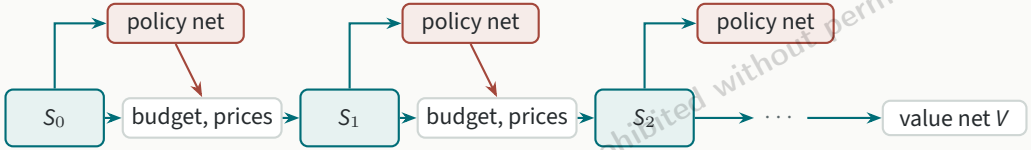
Method 1: DeepHAM

A network inside an average learns the moments that matter, and the policy is trained on simulated paths



© Zhigang Feng
Redistribution prohibited without permission

DeepHAM in One Picture: an Unrolled Panel



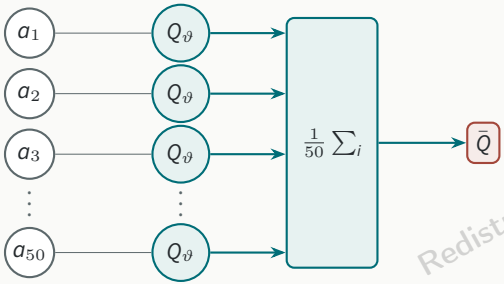
S_t: all 50 households' (a_i, ε_i) and z_t; prices from $K_t = \frac{1}{50}\sum_i a_i$

© Zhigang Feng. Redistribution prohibited without permission

- 384 economies run side by side, each a panel of $N = 50$ households.
- The actor simulates 150 periods forward from a stored cross-section inside the loss; a critic values what is left. Where the stored cross-sections come from: the training-loop pages.
- **Both networks read the panel through generalised moments.**

Source: Method 1 follows Han, Yang & E, "DeepHAM: A Global Solution Method for Heterogeneous Agent Models with Aggregate Shocks", *QE* 17(2), 2026 (arXiv 2112.14377), and its public repository (Figure 2, §3). At CPU scale in PyTorch: lab block D.

Generalised Moments: a Network Inside an Average



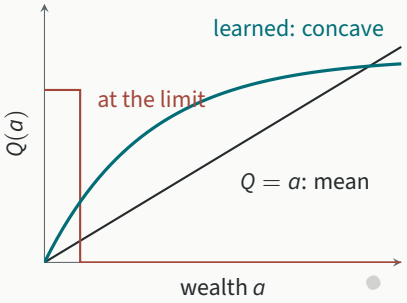
one network, shared by all:
2 layers of 12 tanh units

- $\bar{Q}_t = \frac{1}{N}\sum_i Q_\vartheta(a_t^i)$: one small network, applied to each household's wealth, then averaged.
- Household i's policy reads $(a_i, \varepsilon_i, z_t, \bar{Q}_t)$.
- ϑ is trained with the policy and value networks: the moment is whatever lowers the loss.
- **Krusell–Smith run: $J = 1$, one generalised moment of wealth.**

Source: The paper, §2.2; repository *util.py*; configurations *game_nn_n50_0fm1gm.json* ($J = 1$) and *game_nn_n50.json* (the mean), identical in everything else.

© Zhigang Feng
Redistribution prohibited without permission

Why an Average, and What It Nests

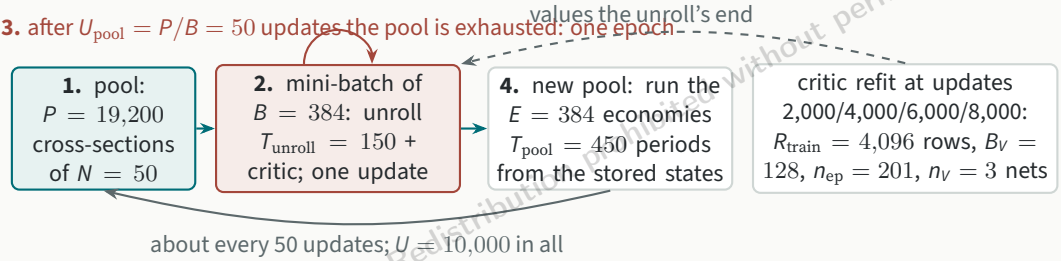


- **Permutation invariance**: renumbering households leaves $\sum_i Q(a_i)$ unchanged.
- $Q(a) = a$ gives the mean, Krusell and Smith's state; an indicator of the limit gives the hand-to-mouth share.
- Learned here: Q concave in wealth, value linear in $\bar{Q}$. Moving wealth from rich to poor raises $\bar{Q}$ at the same K.
- **Bellman error: the Krusell–Smith rule 0.0253; DeepHAM with the mean 0.0184; with one generalised moment 0.0151.**

© Zhigang Feng
Redistribution prohibited without permission

Source: The paper, §3.2, Table 2, Figure 3. Curves schematic.

DeepHAM's Training Loop in One Picture

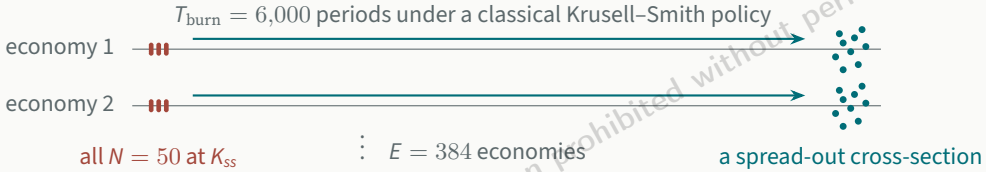


© Zhenhuan Cheng Redistribution prohibited without permission

- **1–2.** A pool of $P = E \times n_{\text{snap}} = 19{,}200$ cross-sections of $N = 50$; a mini-batch is $B = 384$ of them. **3–4.** After $U_{\text{pool}} = P/B = 50$ updates the pool is exhausted; a new pool from the stored initial states.
- **The $T_{\text{unroll}} = 150$-period unroll is inside the loss: it moves nothing in the pool.**

Source: Repository *train_KS.py*, *policy.py* (*train*, *sampler*), *dataset.py*; the Krusell–Smith configuration. Every symbol is collected on this method's last page.

1. Samples: 384 Economies of 50, Burned In



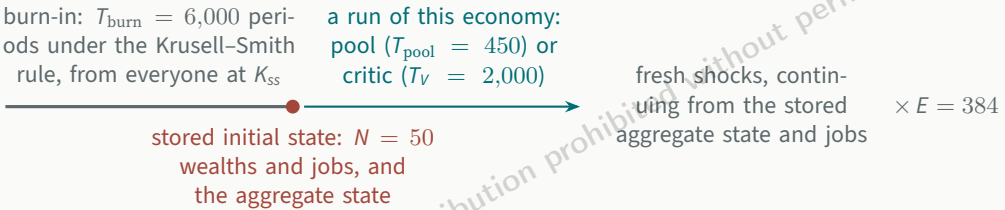
Redistribution prohibited without permission

- $E = 384$ economies side by side, each with $N = 50$ households and its own aggregate shocks.
- All start at K_{ss}; employment drawn at the aggregate state's unemployment rate.
- After $T_{\text{burn}} = 6{,}000$ periods, each economy's final cross-section is stored as its **initial state**.
- **A "distribution" is a sample of $N = 50$ households; the algorithm holds $E = 384$ of them.**

© Zhigang Feng

Source: Repository *dataset.py* (*KSInitDataSet*), *simulation_KS.py*, the Krusell–Smith configuration.

1. Where Every Run Starts: the Stored Initial State

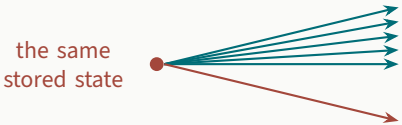


© Zhigang Feng Redistribution prohibited without permission

- After the burn-in each of the $E = 384$ economies has a **stored initial state**: its $N = 50$ households' wealth and employment, and the aggregate state.
- Every later run of that economy, a pool run of $T_{\text{pool}} = 450$ or a critic run of $T_V = 2{,}000$, starts from its stored state, with fresh shocks.
- **A run starts from the stored state. What moves the stored state: the next two pages.**

Source: Repository *dataset.py* (*update_with_burn*, *update_from_simul*), *simulation_KS.py* (*simul_k*, *simul_shocks*).

1. Before the First Refit: Every Run Restarts From the Same State



pool runs, $T_{\text{pool}} = 450$ each, fresh shocks: about one per $U_{\text{pool}} = 50$ updates, about 40 before the first refit

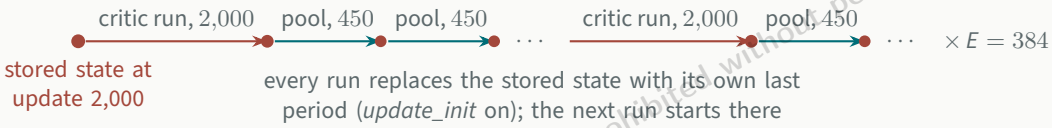
the first critic run, $T_V = 2{,}000$, before any update, under the constant consumption share 0.4

© Shigang Peng. Redistribution or hosting without permission

- For updates 0–1,999 the stored states stay the burned-in ones: each pool run restarts from the same state, with new shocks.
- The first critic run, before any update, starts there too; its rows are the first critic fit's data.
- **About 40 pool runs from one state, with different shocks: nothing accumulates yet.**

Source: Repository *policy.py* (*train*: *update_init* False until the first refit), *dataset.py* (*get_policydataset*, *get_valuedataset*).

1. After the First Refit: Runs Join End to End

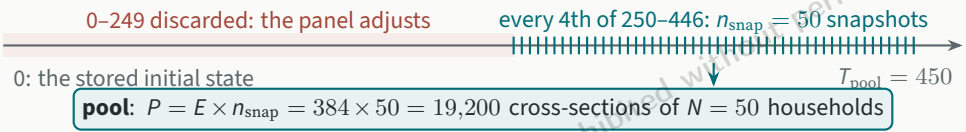


- At update $F = 2{,}000$ the critic run replaces each economy's stored state with its own last period, and so does every run after it, pool or critic.
- From then on each economy is one long simulation under the evolving rule: about 80,000 periods by update $U = 10{,}000$.
- **The same $E = 384$ economies feed the actor's pool and the critic's data; each datum is a cross-section of $N = 50$.**

Source: Repository *policy.py* (*train*: *update_init* set at the first refit, never reset), *dataset.py* (*update_from_simul*). Lengths not to scale.

© Jihgang Fu Redistribution prohibited without permission

2–3. Actor: Build the Pool, Draw Mini-Batches, Exhaust It, Rebuild



One pool, then $U_{\text{pool}} = 50$ updates

1. build the pool: E economies run $T_{\text{pool}} = 450$ periods; keep every 4th of 250–446: $P = 19{,}200$
2. draw a mini-batch: $B = 384$ rows without replacement; fresh shocks for $T_{\text{unroll}} = 150$ periods
3. one Adam update of θ on the unroll loss, rate 4×10^{-4}
4. repeat 2–3; after $U_{\text{pool}} = P/B = 50$ updates every row is used once: *exhausted*, back to 1

- $U = 10{,}000$ **updates in all, one per mini-batch, about 200 pools. The pool supplies starting states, not targets.**

Source: Repository *dataset.py* (*get_policydataset*, *crazyshuffle*, *next_batch*) and *policy.py* (*sampler*, *train*); *epoch_resample* $= 0$ in the configuration.

© Zhiyang Feng. Redistribution prohibited without permission

4. Three Clocks, and the Algorithm

clock	every	what is simulated, from where, and what it feeds
actor update	each mini-batch of B	$T_{\text{unroll}} = 150$ periods inside the loss, then discarded; one Adam step on θ
pool rebuild	$U_{\text{pool}} \approx 50$ updates	$T_{\text{pool}} = 450$ periods of all E from the stored states; the next 50 mini-batches
critic refit	$F = 2{,}000$ updates	$T_V = 2{,}000$ periods of all E from the stored states, then moved; $R = 4{,}608$ rows, $n_V = 3$ nets

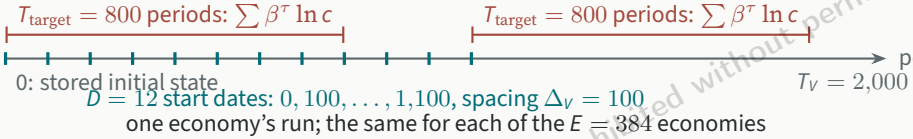
The algorithm: $U = 10{,}000$ actor updates

1 burn in the $E = 384$ economies for T_{burn}; at the constant share 0.4: critic run, fit, pool run
2 for $U = 10{,}000$ updates: a mini-batch of B, unroll T_{unroll}, one Adam step on θ
3 about every $U_{\text{pool}} = 50$ updates: rebuild the pool from the stored states
4 at updates 2,000/4,000/6,000/8,000: a critic run and a refit; the stored states move from now on

Source: Repository *train_KS.py*, *policy.py* (*train*, *sampler*), *dataset.py* (*update_from_simul*); *num_step*, *freq_update_v* in the configuration.

© Yiqing Xing Redistribution prohibited without permission

The Critic's Data: One Economy's 2,000 Periods, 12 Dates; Times 384

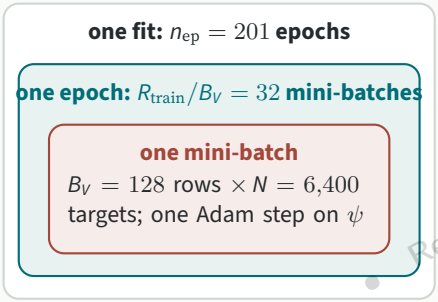


© Zhigang Feng. Redistribution prohibited without permission

- **Rows**: the figure is one economy's run; all $E = 384$ run $T_V = 2{,}000$ periods from their stored states. The cross-sections at the $D = 12$ dates are the rows: $R = D \times E = 4{,}608$.
- Why 12: a date needs $T_{\text{target}} = 800$ future periods ($1{,}100 + 800 < 1{,}999$); dates $\Delta_V = 100$ apart differ.
- **Samples**: a row keeps its $N = 50$ households, each with its own realised T_{target}-period target: $R \times N = 230{,}400$.
- $R_{\text{valid}} = 512$ **rows set aside to check the fit,** $R_{\text{train}} = 4{,}096$ **trained on. The critic's data are its own runs, not the actor's pool.**

Source: Repository *dataset.py* (*get_valuedataset*, *process_vdatadict*); *value_config* in the Krusell–Smith configuration.

The Critic's Fit: 32 Mini-Batches an Epoch, 201 Epochs, Three Nets



$U_{\text{fit}} = 201 \times 32 = 6{,}432$ updates per net; $\times\, n_V = 3$ nets, $\times\, n_{\text{fit}} = 5$ fits

- **Epoch**: one pass over the $R_{\text{train}} = 4{,}096$ rows in mini-batches of $B_V = 128$: 32 updates of ψ.
- **Fit**: $n_{\text{ep}} = 201$ passes over the *same* rows, reshuffled each pass: $U_{\text{fit}} = 6{,}432$ updates.
- **Three nets**: $n_V = 3$, same rows, different starts, averaged by the actor.
- **Fits**: once before training, then at 2,000, 4,000, 6,000, 8,000 ($n_{\text{fit}} = 5$); *frozen* between.
- **Tolerable: the critic carries only $\beta^{149} \approx 0.22$ of the objective, and a refit re-evaluates the current rule in full.**

Source: Repository *value.py* (*train*), *policy.py* (*train*, *freq_update_v*); *num_epoch*, *num_vnet*, *batch_size* in the configuration.

© Zhigang Feng Redistribution prohibited without permission

5a. Inside One Actor Update: the Loss

- **Unroll**: simulate $T_{\text{unroll}} = 150$ periods forward *inside the loss*, from each S_0 in the mini-batch $\mathcal{B}$ of $B = 384$, so the gradient runs back through every period:

$$\max_{\theta} \frac{1}{B} \sum_{S_0 \in \mathcal{B}} \Big[\sum_{t=0}^{148} \beta^t \ln c_t^1 + \beta^{149} \bar{V}(S_{149}) \Big], \qquad c_t^i = \sigma\big(\mathcal{C}_\theta(a_i, \varepsilon_i, z_t, \bar{Q}_t)\big) \cdot \text{wealth}_t^i .$$

- $\bar{V}$ is the average of the $n_V = 3$ critics; $\beta^{149} \approx 0.22$ leaves the rest of the future to them. One Adam update of θ per mini-batch: $U = 10{,}000$ in all, rate 4×10^{-4}, constant.
- Only household 1 optimises; the other 49 use the same network with the gradient stopped, and set prices and $\bar{Q}$: **fictitious play**.
- **Not 384 cross-sections $\times$ 50 households per step: the reshuffle only makes household 1 a random member. Averaging all 50 would be a planner's problem.**

© Zhigang Feng redistribution prohibited without permission

Source: The paper, eq. (10), Algorithm 1; repository *policy.py* (*loss*, the "socialplanner" option).

5b. Inside One Critic Update: the Loss

- A mini-batch $\mathcal{B}$ is $B_V = 128$ training rows (s, e), a date and an economy; every household in each counts:

$$\mathcal{L}(\psi; \mathcal{B}) = \frac{1}{B_V N} \sum_{(s,e)\in\mathcal{B}} \sum_{i=1}^{N} \left[V_\psi\left(a^e_{i,s}, \varepsilon^e_{i,s}, z^e_s, \bar{Q}^{V,e}_s\right) - \widehat{V}^e_{i,s} \right]^2, \qquad \widehat{V}^e_{i,s} = \sum_{\tau=0}^{T_{\text{target}}-1} \beta^\tau \ln c^e_{i,s+\tau}.$$

- One Adam update of ψ per mini-batch, rate 10^{-4}: 32 per epoch, $U_{\text{fit}} = 6{,}432$ per fit, $n_{\text{fit}} = 5$ fits per net.
- A row keeps its $N = 50$ households together: each household's input includes $\bar{Q}^{V,e}_s$, the moment of its own cross-section.
- $\beta^{T_{\text{target}}} = \beta^{800} < 3.3 \times 10^{-4}$: the truncated sum stands in for the infinite one.
- **Monte Carlo evaluation, as in Session 3: each target is one realised future, and the regression averages the noise.**

Source: The paper, eq. (8)–(9); repository *dataset.py*, *value.py*.

© Thiago Ferra · Redistribution prohibited without permission

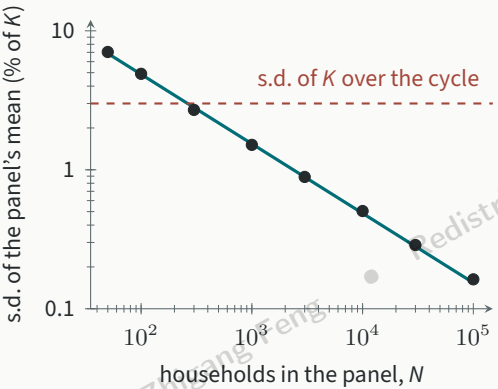
The Symbols

symbol	value	what it counts, and the formula
N, E	50, 384	households in one economy; economies run side by side
T_{burn}, T_{pool}	6,000, 450	burn-in periods, once; periods of one pool run
n_{snap}, P	50, 19,200	snapshots kept per pool run; the pool, $P = E \times n_{\text{snap}}$
B, T_{unroll}	384, 150	actor mini-batch ($B = E$ here); periods unrolled inside the loss
U, U_{pool}	10,000, 50	actor updates in all; updates one pool serves, $U_{\text{pool}} = P/B$
T_V, T_{target}, Δ_V	2,000, 800, 100	a critic run; a target's horizon ($\beta^{T_{\text{target}}} < 3.3 \times 10^{-4}$); date spacing
D, R	12, 4,608	start dates; rows of critic data, $R = D \times E$, each with N targets
R_{valid}, R_{train}	512, 4,096	rows set aside to check the fit; rows trained on
B_V, n_{ep}	128, 201	critic mini-batch in rows; epochs per fit; $R_{\text{train}}/B_V = 32$ an epoch
U_{fit}, n_V	6,432, 3	updates per fit per net, $n_{\text{ep}} \times 32$; nets averaged
F, n_{fit}, U_V	2,000, 5, 96,480	updates between refits; fits per net; critic updates, $n_{\text{fit}}\, n_V\, U_{\text{fit}}$

© Zhigang Feng · Reuse by permission only · Do not distribute without permission

Source: Repository configurations *game_nn_n50_0fm1gm.json* and *game_nn_n50.json* (identical but for the moment), *dataset.py*, *policy.py*, *value.py*.

What 50 Households Cost



- Wealth has s.d. 18.9 around $K = 38.8$: a panel's mean misses K by $18.9/\sqrt{N}$.
- $N = 50$: 2.67, 2.3 times the s.d. of K over the whole cycle.
- The paper treats $N = 50$ as the model, with similar results at 100 and 200.
- **A sampled state, and prices that inherit its noise.**

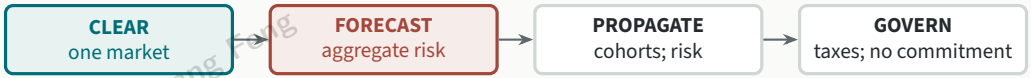
© Zhiyang Peng. Redistribution prohibited without permission

Source: Lab block B, 400 panels per N. Quarterly, as published: $\alpha = 0.36, \beta = 0.99, \delta = 0.025, u = \ln c, z \in \{0.99, 1.01\}$. The paper, fn. 2.

Session 4.2

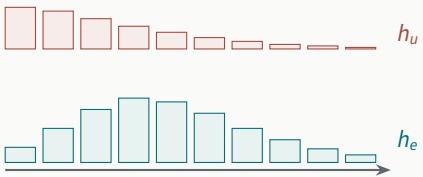
Method 2: Deep Equilibrium Nets

Keep the whole histogram, and ask only that the first-order conditions hold one period ahead



© Zhigang Feng
Redistribution prohibited without permission

The Same Economy, With a Histogram as the State



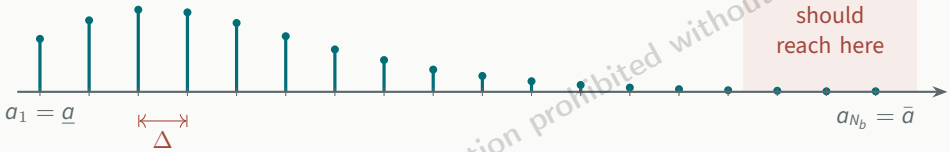
one asset grid of $N_b = 100$ points; a histogram of the unemployed and one of the employed

- State (z, Γ), with $\Gamma = (h_u, h_e)$: two histograms on one asset grid of $N_b = 100$ points, $2N_b = 200$ masses.
- Two, because employment is persistent: who is poor today predicts who is unemployed tomorrow, so assets alone do not determine next period.
- One network: $(\varepsilon, a; z, \Gamma) \mapsto (a', \lambda)$; prices from K.
- **Unlike DeepHAM: every mass is an input; no learned moment, no critics.**

© Zhigang Feng
Redistribution prohibited without permission

Source: Method 2 follows Azinovic, Gaegauf & Scheidegger, "Deep Equilibrium Nets", *IER* 63(4), 2022, and its public continuum-of-agents code, adapted to Krusell–Smith. Here: App. A.5; the code's state. A mean-capital version: lab block E.

Encoding the Distribution: Mass on a Grid of Points

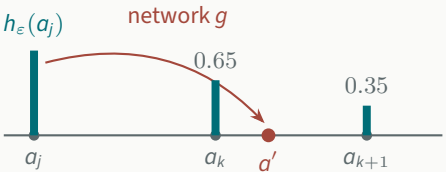


© Eduardo Rossi
No distribution prohibited without permission

- N_b *equidistant* points: the first is the limit $\underline{a}$; the last, $\bar{a}$, high enough that little mass reaches it. Savings above it are set to it.
- The code: $N_b = 100$ points on $[\underline{a}, \bar{a}] = [0, 20]$: $\Delta = \bar{a}/(N_b - 1) \approx 0.2$. A mass $h(a_j)$ sits *on* each point: a finite vector the network can read, and move exactly.
- **Unlike DeepHAM: the distribution is written on a grid, not sampled by 50 households.**

Source: The paper, following Reiter (2009) and Young (2010); the continuum-of-agents code (the asset grid).

From Today's Histogram to Tomorrow's: One Lottery Step



For each ε and grid point a_j

1. saving $a' = g(\varepsilon, a_j; z, \Gamma)$, set to a_{N_b} if above
2. a' lies between a_k and a_{k+1}
3. weights $w_k = 1 - |a_k - a'|/\Delta$, $w_{k+1} = 1 - w_k$
4. send $w_k h_\varepsilon(a_j)$ to a_k, $w_{k+1} h_\varepsilon(a_j)$ to a_{k+1}

$$h'_{\varepsilon'}(a_k) = \sum_{\varepsilon} \pi(\varepsilon' \mid \varepsilon; z, z') \sum_{j} h_\varepsilon(a_j)\, w_k\big(a'_{\varepsilon j}\big)$$

© Edgard Feres
Redistribution prohibited without permission

- Then employment moves between the two histograms, with $\pi(\varepsilon' \mid \varepsilon; z, z')$: one Γ' for each z'. Mass is conserved; the mean of a' is exact.
- **Unlike DeepHAM: tomorrow's distribution is computed, $2N_b = 200$ network calls, not simulated household by household.**

Source: The continuum-of-agents code (the grid weights and the next-period histogram); Young, *JEDC* (2010).

The Equilibrium Conditions Are the Loss

- At a point $(\varepsilon, a; z, \Gamma)$, with $c = (1 - \delta + r)a + y(\varepsilon) - a'$ and c' from the same network at $(\varepsilon', a'; z', \Gamma')$:

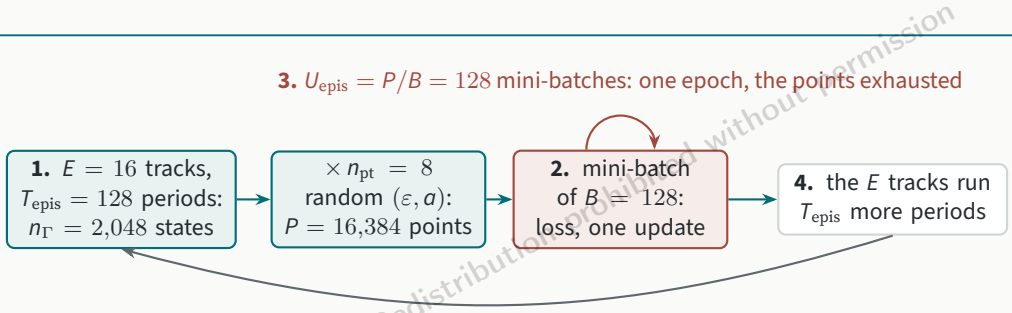
$$R_{EE} = \frac{\left[\beta\, \mathbb{E}\left[(1 - \delta + r')/c'\right] + \lambda\right]^{-1}}{c} - 1, \qquad R_{KKT} = a'\lambda.$$

- $\mathbb{E}$: an exact sum over the four (z', ε'). $R_{EE} = 0$ is the Euler equation; $R_{KKT} = 0$ lets $\lambda > 0$ only at the limit.
- $\Gamma'(z') = (h'_u, h'_e)$: the lottery step of the last page, one for each z'.
- **Unlike DeepHAM: first-order conditions one period ahead, not utility over a 150-period rollout.**

Source: The paper; the continuum-of-agents code (its Euler and KKT terms).

© Thierry Fel — Redistribution prohibited without permission

Deep Equilibrium Nets' Training Loop in One Picture

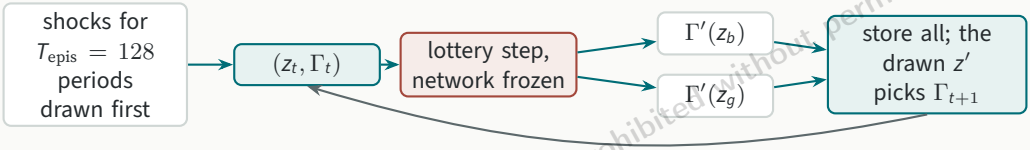


© Zhigang Feng. Redistribution prohibited without permission

- **1. Samples**: $n_\Gamma = 2{,}048$ states $\times\, n_{\text{pt}} = 8$ random (ε, a): $P = 16{,}384$ points.
- **2–4.** Mini-batches of $B = 128$: $U_{\text{epis}} = P/B = 128$ updates exhaust the points, one epoch.
- **No look-ahead in the loss: the successors were stored in the simulation.**

Source: The continuum-of-agents code (the episode loop, training data, batching). Symbols: this method's symbol page.

1. Samples: Simulated, and Stored With Their Successors



© Zhigang Feng · Redistribution prohibited without permission

- **Simulation for data**: $E = 16$ tracks, $T_{\text{epis}} = 128$ periods each per episode; the next page shows where they start and how they continue.
- Each period: one lottery step per z'; all successors stored; the pre-drawn shock picks the next state.
- **Inside the loss**: no simulation; $t + 1$ is a stored successor.
- **A one-period condition at states from a long simulation; DeepHAM simulates 150 periods inside its loss.**

Source: The continuum-of-agents code (shock simulation, the forward pass, the episode loop).

1. Samples: 16 Tracks, 128 Periods an Episode

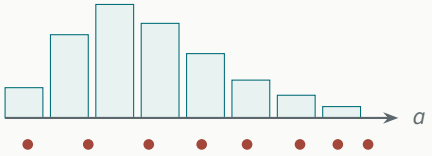


© Zhigang Feng Redistribution prohibited without permission

- Episode 1: $E = 16$ copies of *one* state (wealth near zero), differing only by their shocks.
- Each episode: the code sets $n_\Gamma = 2{,}048$ states, so $T_{\text{epis}} = n_\Gamma / E = 128$ periods per track; at each dashed line a track continues from its own end state.
- **Not 16 hand-picked starts: $E = 16$ simulations of $n_{\text{epis}} T_{\text{epis}} \approx 8.3$ million periods each, in T_{epis}-period pieces, never restarted.**

Source: The continuum-of-agents code (starting states, shock simulation, the episode loop). The paths are schematic.

1. Samples: Each State Paired With 8 Evaluation Points



one stored (z, Γ), paired with $n_{\text{pt}} = 8$ random (ε, a): 8 evaluation points, not 8 agents

- **Data**: the $n_\Gamma = E\,T_{\text{epis}} = 2{,}048$ states (z, Γ) just simulated, with their stored successors.
- **Evaluation points**: each state (z, Γ) is paired with $n_{\text{pt}} = 8$ household states (ε, a), drawn at random: a uniform on $[\underline{a}, \bar{a}]$, ε a fair coin. $P = n_\Gamma\, n_{\text{pt}} = 2{,}048 \times 8 = 16{,}384$ points $(\varepsilon, a; z, \Gamma)$.

- (z, Γ) is not a starting point, and the 8 are not agents: the economy is a continuum. It is where the loss is evaluated, with its stored successors $\Gamma'(z')$.
- Why points: the network is pointwise, $(\varepsilon, a; z, \Gamma) \mapsto (a', \lambda)$; the Euler condition is a household's. Why uniform: the lottery uses the policy at all $2N_b = 200$ grid points.

Source: The continuum-of-agents code (training-data construction).

© Zhiheng Huang • Redistribution prohibited without permission

2–3. Draw Mini-Batches, Exhaust the Points, Simulate Again

One episode's points, then $U_{\text{epis}} = 128$ updates

1 the points: $n_\Gamma = 2{,}048$ states $\times\, n_{\text{pt}} = 8$ random (ε, a): $P = 16{,}384$, shuffled
2 draw a mini-batch $\mathcal{B}$: $B = 128$ points, without replacement
3 one Adam update of θ on the loss of the next page, rate 10^{-5}
4 repeat 2–3; after $U_{\text{epis}} = P/B = 128$ updates every point is used once: *exhausted*, and discarded
5 the $E = 16$ tracks run $T_{\text{epis}} = 128$ more periods, network frozen: new points, back to 1

- **Epoch**: one pass over the points, U_{epis} updates: an episode is one epoch. $U_{\text{epis}} = T_{\text{epis}}$ because each simulated period yields $E\, n_{\text{pt}} = 128 = B$ points: one mini-batch.
- A point is used once and never again; DeepHAM's critic reuses its rows for 201 passes.
- **One Adam update of θ per mini-batch: $U = n_{\text{epis}}\, U_{\text{epis}} = 65{,}000 \times 128 \approx 8.3$ million in all. The points are evaluation points, not starting states.**

Source: The continuum-of-agents code (training data, shuffling and batching, the episode loop); *epochs_per_episode* $= 1$.

© Zhangang Tang · Redistribution prohibited without permission

5. Inside One Update: the Loss

- A mini-batch $\mathcal{B}$ is $B = 128$ evaluation points $(\varepsilon, a; z, \Gamma)$; each gives the Euler and KKT residuals R_{EE}, R_{KKT} defined above:

$$\mathcal{L}(\theta; \mathcal{B}) = \frac{1}{B} \sum_{(\varepsilon,a;z,\Gamma)\in\mathcal{B}} \Big[R_{EE}^2 + R_{KKT}^2\Big], \qquad \text{one Adam step at rate } 10^{-5}.$$

- The successors $\Gamma'(z')$ were stored during simulation: the network is re-evaluated at $(\varepsilon', a'; z', \Gamma')$, but how Γ moves is not differentiated.
- One period ahead only: the long simulation chooses the states; the loss never looks past $t + 1$.
- **Unlike DeepHAM: no rollout inside the loss and no critic; the gradient sees one period, at stored states.**

Source: The continuum-of-agents code (the loss and the training step).

© Zhiyang Feng · Redistribution prohibited without permission

The Symbols

symbol	value	what it counts, and the formula
N_b, Δ	100, 0.2	grid points on $[\underline{a}, \bar{a}] = [0, 20]$; their spacing, $\bar{a}/(N_b - 1)$
E	16	tracks run side by side, identical at the start
n_Γ, T_{epis}	2,048, 128	states (z, Γ) simulated per episode (the code's setting); periods per track, n_Γ / E
n_{pt}, P	8, 16,384	random (ε, a) per state (2×4), not agents; evaluation points, $P = n_\Gamma \, n_{\text{pt}}$
B, U_{epis}	128, 128	mini-batch; updates per episode, $U_{\text{epis}} = P/B$ $(= T_{\text{epis}}$: $E \, n_{\text{pt}} = B)$
n_{epis}, U	65,000, 8,320,000	episodes; updates in all, $U = n_{\text{epis}} \, U_{\text{epis}}$
$n_{\text{epis}} \, T_{\text{epis}}$	8,320,000	periods each track is simulated over training

- All are settings of the public code: choices, not properties of the model.
- **One episode: simulate n_Γ states, pair them into P evaluation points, take U_{epis} steps, discard the points, move on.**

Source: The continuum-of-agents code: *num_tracks*, *len_episodes*, *num_id_per_shock*, *batch_size*, *num_episodes*, *nA*, *lr*.

© Yongcheng Feng · Redistribution prohibited without permission

Two Clocks, and the Algorithm

clock	every	what is simulated, from where, and what it feeds
network update	each mini-batch of B points	nothing simulated: one Euler and one KKT residual per point, at its stored successors; one Adam step on θ
episode	$U_{\text{epis}} = 128$ updates	$T_{\text{epis}} = 128$ periods of all E tracks from their own end states, network frozen; $P = 16{,}384$ fresh points replace the old

The algorithm: $n_{\text{epis}} = 65{,}000$ episodes, $U \approx 8.3$ million updates

1 $E = 16$ tracks, identical at the start: a state (z, Γ), two N_b-point histograms
2 for each of $n_{\text{epis}} = 65{,}000$ episodes:
3 $T_{\text{epis}} = 128$ periods per track, network frozen; each period a lottery step per z', all stored
4 the $n_\Gamma = 2{,}048$ states $\times$ $n_{\text{pt}} = 8$ random (ε, a): P points; $U_{\text{epis}} = P/B = 128$ Adam steps

- **Two levers against DeepHAM: an exact state, not a sample; one period, not a long rollout.**

Source: The paper; the continuum-of-agents code (the episode loop; *num_episodes*, *epochs_per_episode*).

© Zhigang Feng · Redistribution prohibited without permission

Session 4.2

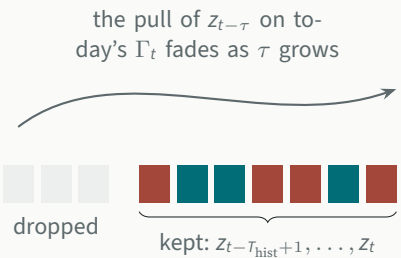
Method 3: The Sequence Space

Forget the distribution: the recent shocks already determine it



© Zhigang Feng Redistribution Prohibited without permission

The Distribution Is a Function of Past Shocks

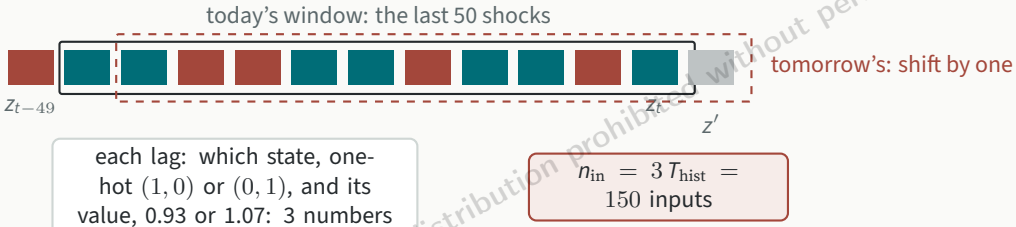


- Given the start and the policy, today's distribution is a function of all past shocks: $V(a, \varepsilon; \Gamma_t, z_t) = V(a, \varepsilon; z_0, \ldots, z_t)$.
- **Memory loss**: old shocks fade, so keep the last T_{hist}: $V \approx V(a, \varepsilon; z_{t-T_{\text{hist}}+1}, \ldots, z_t)$.
- The input is *exogenous*: the shock process sets its distribution, whatever the policy does.
- **Why it works: history $\to \Gamma$ is the law of motion iterated under one fixed policy. The tutorial keeps $T_{\text{hist}} = 50$.**

© Zhigang Feng
Redistribution prohibited without permission

Source: Method 3 follows Azinovic-Yang & Žemličkа, "Deep Learning in the Sequence Space", working paper, arXiv 2509.13623 (2025), with their slides (2026) and tutorial notebook; that notebook in PyTorch: the Session 4 lab, block F.

Encoding the Input: a Window of 50 Shocks

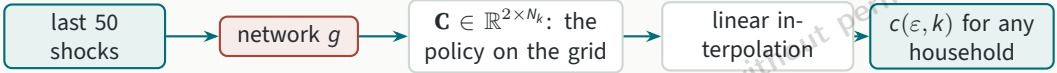


Redistribution prohibited without permission

- Stored as all $T_{\text{hist}} = 50$ one-hot pairs, then all 50 values: $n_{\text{in}} = 150$ numbers that stand in for the aggregate state, the distribution included. For tomorrow, drop the oldest shock and append z'.
- **Unlike DeepHAM: no household state and no moment enters; the input is exogenous, its distribution fixed by the shock process.**

Source: The tutorial notebook (history encoding and rolling windows).

Operator Learning: History In, a Whole Policy Out

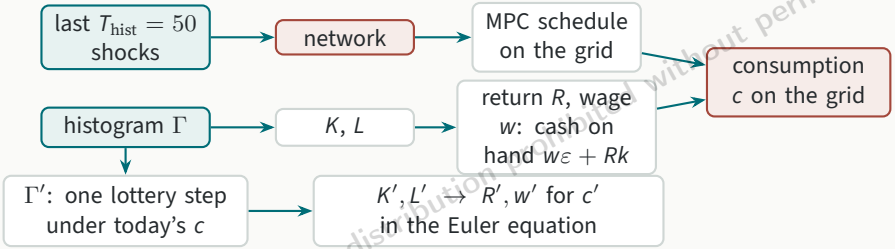


- **Operator learning** (Zhong, 2023; the paper extends it): write the policy as $f(X, x) = g(X)(x)$. The network g maps the aggregate input, here the history, to a whole *function* of the individual state $x = (\varepsilon, k)$.
- The function is stored as its values on a fixed grid, and read between grid points by linear interpolation. One call gives every household's policy.
- Shape is imposed at the grid points: the output is a starting share $\alpha(\varepsilon) < 1$ and falling MPCs, so c is increasing, concave and leaves $k' \geq 0$; linear interpolation keeps that shape between points.
- **Unlike DeepHAM, whose network is evaluated household by household: here the network's output is the policy function itself.**

Source: Zhong, Y. (2023), "Operator Learning in Macroeconomics", working paper; the tutorial notebook.

© Yingguang Zhong. Distribution prohibited without permission

Where the Distribution Still Enters: Through Prices

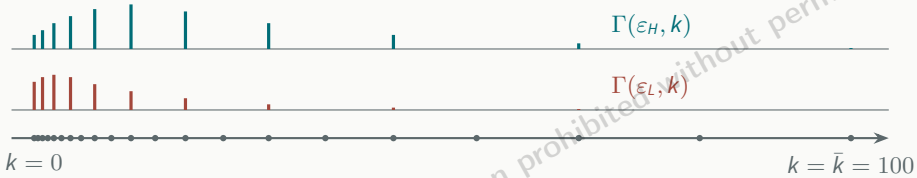


Redistribution prohibited without permission

- The network reads only shocks and gives the *shape* of consumption; the *level* needs today's prices, from Γ. Tomorrow's prices need Γ', one lottery step on.
- So every state is a pair (last $T_{\text{hist}} = 50$ shocks, Γ), with Γ simulated along those same shocks. Even a one-period loss needs a starting Γ.
- **The network reads shocks; the distribution sets the prices it is evaluated at.**

Source: The tutorial notebook (the network's forward pass takes the history and today's prices; the loss).

Why a Log-Spaced Grid



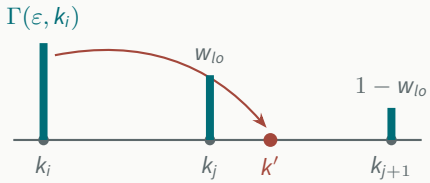
one grid for the policy's values and for the histogram (drawn schematically)

© Thomas Renault — Redistribution prohibited without permission

- The grid is where the policy function lives. Linear interpolation errs where the function bends, and consumption bends most near the borrowing limit.
- So points are dense there: $N_k = 101$ points equally spaced in $\log(k + 0.5)$ on $[0, \bar{k}]$: $\Delta x \approx 0.053$, 0.03 apart near zero, 5 at the top. The paper suggests log-log spacing.
- **The histogram sits on the same grid: the lottery moves mass straight from the policy's own grid values.**

Source: The paper's section on operator learning; the tutorial notebook (capital grid).

The Same Lottery, on the Log Grid



For each ε and grid point k_i

1. $k' = w\varepsilon + Rk_i - c(\varepsilon, k_i)$, clipped to $[0, \bar{k}]$
2. bracket: $j = \lfloor (\log(k' + 0.5) - x_0)/\Delta x \rfloor$
3. $w_{lo} = (k_{j+1} - k')/(k_{j+1} - k_j)$
4. send $w_{lo}\Gamma(\varepsilon, k_i)$ to k_j, the rest to k_{j+1}

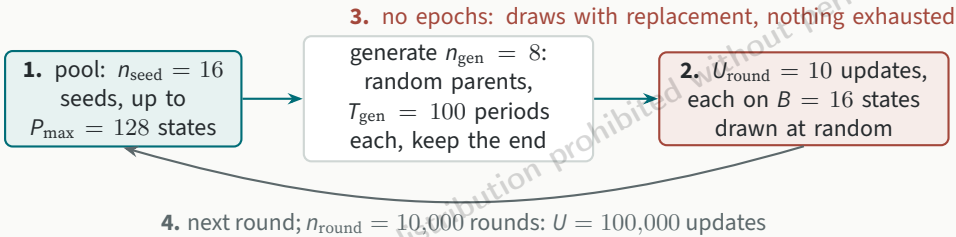
$$\Gamma' = \Pi_\varepsilon^\top \, \Gamma_{\text{after}}, \qquad K' = \sum_{\varepsilon,k} k\, \Gamma'(\varepsilon, k), \qquad L' = \sum_{\varepsilon,k} \varepsilon\, \Gamma'(\varepsilon, k).$$

© Shugang Peng. Redistribution prohibited without permission

- Identical in spirit to Method 2's step; only the grid differs. In the tutorial, Π_ε does not depend on z.
- **What differs from Deep Equilibrium Nets is who reads Γ': there the network does; here only K' and L' do.**

Source: The tutorial notebook (the distribution step); Young, *JEDC* (2010).

The Sequence Space's Training Loop in One Picture



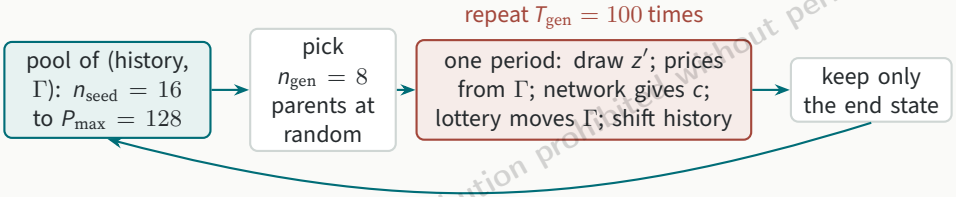
Redistribution prohibited without permission

- **1. Samples**: $n_{\text{gen}} = 8$ states a round, run $T_{\text{gen}} = 100$ periods from random parents; pool $\leq P_{\max} = 128$.
- **2–4.** $B = 16$ drawn with replacement per update; $U_{\text{round}} = 10$ a round, $n_{\text{round}} = 10{,}000$ rounds: $U = 100{,}000$.
- **The loss looks one period ahead (both z'): it moves nothing in the pool.**

© Zhuoyi Peng

Source: The tutorial notebook (buffer, parent draws, training loop). Every symbol is collected on this method's symbol page.

1. Samples: One Round Adds 8 States

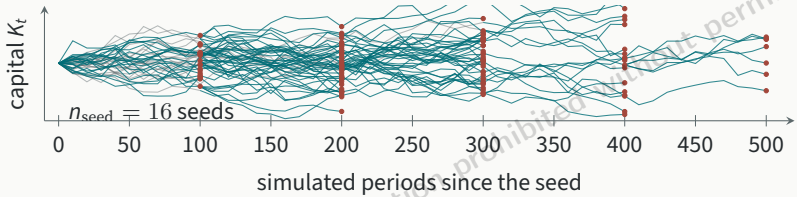


© Thiago Ferreira — Redistribution prohibited without permission

- **Pool**: $n_{\text{seed}} = 16$ seeds (one all-low history, Γ_0), $+ n_{\text{gen}}$ a round: 16, 24, …, full at $P_{\max} = 128$ from round $(P_{\max} - n_{\text{seed}})/n_{\text{gen}} = 14$; then the oldest n_{gen} leave.
- **Parents**: a run starts at a parent drawn at random, *with replacement*, from the pool as it stands; a parent can be continued twice, and the seeds are the first parents.
- **Unlike DeepHAM:** $P_{\max} = 128$ **whole economies in all, renewed** $n_{\text{gen}} = 8$ **a round.**

Source: The tutorial notebook (seed, continuation, pool update).

1. Samples: a Branching Pool

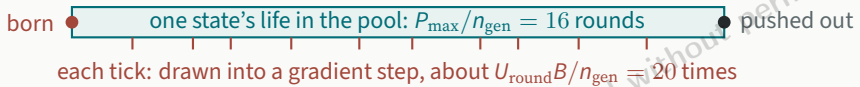


© Yuxiang Feng • Redistribution prohibited without permission

- Each round: $n_{\text{gen}} = 8$ parents drawn with replacement; a child is its parent run $T_{\text{gen}} = 100$ periods on fresh shocks. Red: the pool after 20 rounds; grey: gone.
- **Depth**: a lineage gains T_{gen} a generation, and a parent's age averages 8.5 rounds (uniform over $P_{\max}/n_{\text{gen}} = 16$); after $n_{\text{round}} = 10{,}000$ rounds: $n_{\text{round}} T_{\text{gen}}/8.5 \approx 120{,}000$ periods.
- **Method 2 runs $E = 16$ fixed tracks; here tracks split and die out: a tree's tips.**

Source: The tutorial notebook (seeding, parent draws, continuation, pool update); the depth: a replay of that rule. Paths schematic.

2–4. Mini-Batches Drawn at Random; No Epochs; 10,000 Rounds



- **Mini-batch**: each update draws $B = 16$ states at random, *with replacement*, from the pool as it stands.
- **No epochs**: nothing is used up in order, nothing exhausted; a state stays $P_{\max}/n_{\text{gen}} = 16$ rounds and is drawn about $U_{\text{round}}B/n_{\text{gen}} = 20$ times (above).
- **Rounds**: $n_{\text{round}} = 10{,}000$: $n_{\text{gen}}n_{\text{round}} = 80{,}000$ states from $80{,}000 \times T_{\text{gen}} = 8$ million simulated periods, $P_{\max} = 128$ held at a time; $UB = 1.6$ million state-draws.
- **One Adam update of θ per mini-batch, $U_{\text{round}} = 10$ a round: $U = U_{\text{round}}\, n_{\text{round}} = 100{,}000$ in all, rate 5×10^{-5}, clipped at 1. Every state was simulated under a recent network.**

© Zhigang Feng · Unauthorized distribution prohibited without permission

Source: The tutorial notebook (training loop). The paper instead moves 4,096 pairs one period per episode.

Two Clocks, and the Algorithm

clock	every	what is simulated, from where, and what it feeds
network update	each mini-batch of $B = 16$ states	one period ahead (both z'), discarded; prices and Γ' fixed; one Adam step on θ
round	$U_{\text{round}} = 10$ updates	$T_{\text{gen}} = 100$ periods from $n_{\text{gen}} = 8$ parents drawn from the pool, network frozen; end states join, the oldest leave

The algorithm: $U = U_{\text{round}}\, n_{\text{round}} = 100{,}000$ **updates**

1 seed the pool: $n_{\text{seed}} = 16$ copies of (an all-low history, Γ_0)
2 for $n_{\text{round}} = 10{,}000$ rounds:
3 draw $n_{\text{gen}} = 8$ parents with replacement, run each $T_{\text{gen}} = 100$ periods, keep the end states
4 $U_{\text{round}} = 10$ times: draw $B = 16$ states with replacement, one Adam step on the FB loss

- **No separate burn-in, no epoch, nothing exhausted: the pool is a moving window of the last $P_{\max}/n_{\text{gen}} = 16$ rounds; DeepHAM burns in and exhausts each pool.**

Source: The tutorial notebook (training loop); *N_EPOCHS*: it calls a round an epoch.

© Lizhang Feng · Redistribution prohibited without permission

5. Inside One Update, Line by Line

One Adam step, of $U = 100{,}000$

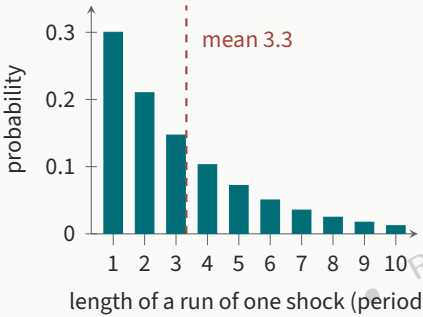
1. draw $B = 16$ states (history, Γ) at random, with replacement, from the pool
2. for each: R, w from Γ; one call $\mathbf{C} = g(\text{history}; R, w) \in \mathbb{R}^{2\times N_k}$; $k'_{ij} = w\varepsilon_i + Rk_j - C_{ij}$
3. Γ' = one lottery step of Γ under c; then K', L', held fixed
4. for each z': $\mathbf{C}' = g(\text{shifted history}; R', w')$, read at each k'_{ij} by interpolation
5. $q = \sum_{z',\varepsilon'} \pi\, \beta R'/c'$; then the Euler gap $e = 1/(qc) - 1$, $s = k'/c$, $\mathrm{FB}(e, s)$
6. loss = mean of FB^2 over $B \times 2N_k = 16 \times 202$; one Adam step, rate 5×10^{-5}, clipped at 1

- One network, one call per state: its output is the whole consumption function on the grid, $c(\varepsilon_i, k_j) = C_{ij}$; not a different network per node.
- The gradient reaches the network through $\mathbf{C}$ and $\mathbf{C}'$ only; prices, Γ and Γ' are held fixed.
- **Unlike DeepHAM: one period, every grid node of every state, no rollout and no critic.**

Source: The tutorial notebook (the loss and the training step).

© Thiago Trafane Oliveira Santos. Reproduction prohibited without permission

Coverage (1): The Draws Follow the True Shock Process

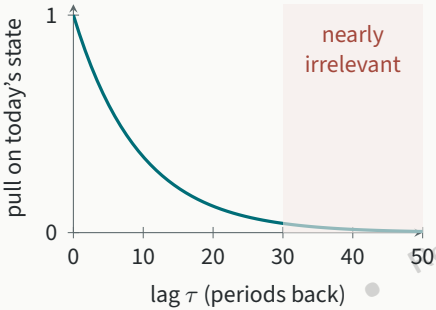


- All $2^{T_{\text{hist}}} = 2^{50} \approx 10^{15}$ histories are possible, and none is ruled out.
- But each new shock is drawn from the true chain: stay with probability 0.7, switch with 0.3. Training therefore meets histories as often as the economy does.
- Runs last $1/0.3 \approx 3.3$ periods on average, so likely histories are a small, structured part of the space.
- **Unlike DeepHAM and Method 2, the history's distribution does not depend on the network; only Γ does.**

© Zhigang Feng Redistribution prohibited without permission

Source: The tutorial notebook (shock process and training loop).

Coverage (2): Old Shocks Matter Little, and the Distribution Comes Along



- **Memory loss**: an old shock's pull on today's state decays geometrically (in the paper's one-sector example, at rate $\max\{|\rho|, \alpha\}$). Histories that differ only far back need nearly the same answer.
- Γ needs no coverage of its own: it is whatever the simulated history produced.

- **Caveats:** each Γ in the pool was simulated under an older network; coverage is checked on simulated paths, not guaranteed; a pool of $P_{\max} = 128$ is a teaching setting.

Source: The tutorial notebook; the paper's argument that a truncated history is approximately sufficient. The decay curve is schematic.

© Zhiming Jiang redistribution prohibited without permission

The Symbols

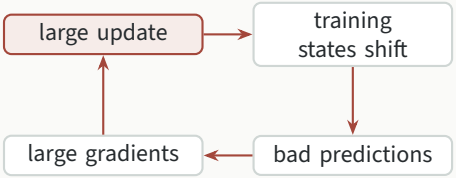
symbol	value	what it counts, and the formula
T_{hist}, n_{in}	50, 150	shocks in the window (the paper's T); inputs, $3\,T_{\text{hist}}$
N_k, $\bar{k}$	101, 100	grid points, equally spaced in $\log(k+0.5)$ on $[0,\bar{k}]$
Δx	0.053	that spacing, $[\log(\bar{k}+0.5)-\log 0.5]/(N_k-1)$; $2N_k = 202$ nodes (ε, k)
n_{seed}, $P_{\max}$	16, 128	seeds; pool capacity, full from round $(P_{\max}-n_{\text{seed}})/n_{\text{gen}} = 14$
n_{gen}, T_{gen}	8, 100	states generated per round; periods each is run
B, U_{round}	16, 10	mini-batch, drawn with replacement; updates per round
n_{round}, U	10,000, 100,000	rounds; updates in all, $U = U_{\text{round}}\, n_{\text{round}}$
$P_{\max}/n_{\text{gen}}$	16	rounds a state stays in the pool
$U_{\text{round}}B/n_{\text{gen}}$	20	times a state is drawn while it is there
$n_{\text{gen}}n_{\text{round}}$, UB	80,000, 1.6 million	states generated; state-draws

© Zhigang Feng. Reproduction prohibited without permission

- R, w, g: the notebook's gross return, wage and network; its μ is the distribution, Γ here.

Source: The tutorial notebook's settings (*N_AGG*, *BUFFER_CAP*, *FB_BATCH*, *FB_STEPS*, *N_EPOCHS*: it calls a round an epoch).

What the Exogenous Domain Buys



state-space training can feed on itself; a history's distribution does not move with the policy

- Reported: stable training across learning rates and seeds, where the state-space version struggles.
- Tutorial: $n_{\text{round}} = 10{,}000$ rounds in 191 CPU-seconds; mean $|\text{FB}| = 1.6 \times 10^{-4}$ over the pool's $P_{\max} = 128$ states. Its economy: annual; ε independent of z.
- The cost: the policy cannot be asked about a distribution no history produced.
- **Unlike the other two: the inputs do not shift when the policy does.**

Source: Stability: their slides (2026), pp. 11, 23–24. Time and $|\text{FB}|$: the tutorial's stored output.

© Zhigang Feng
Redistribution prohibited without permission

Three Pipelines, the Same Letters

	DeepHAM	Deep Equilibrium Nets	Sequence space
simulated	$E = 384$ economies, $N = 50$	$E = 16$ histogram tracks	$\leq P_{\max} = 128$ pool states
a run starts from	the stored state, burned in; moved after the first refit	the track's own end state	a parent drawn from the pool
periods per run	$T_{\text{pool}} = 450$	$T_{\text{epis}} = 128$	$T_{\text{gen}} = 100$
mini-batch B of P	384 of $P = 19{,}200$	128 of $P = 16{,}384$	16 of $P_{\max} = 128$
updates per dataset; datasets	$U_{\text{pool}} = 50$, exhausted; about $U/U_{\text{pool}} = 200$ pools	$U_{\text{epis}} = 128$, exhausted; $n_{\text{epis}} = 65{,}000$ episodes	$U_{\text{round}} = 10$, never exhausted; $n_{\text{round}} = 10{,}000$ rounds
updates in all, U	10,000 (critic 96,480)	8,320,000	100,000
look-ahead	$T_{\text{unroll}} = 150$, simulated	one period, stored	one period, computed

© Shigang Fan · Redistribution prohibited without permission

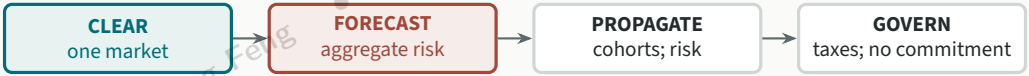
- **Same letters, three values: what differs is what a datum is and how long it lives.**

Source: The three symbol pages of this deck; the opener's questions 3 and 4, in symbols.

Session 4.2

Three Methods Against a Law of Motion

What each method moves forward, and what a government's problem needs



© Zhigang Feng
Redistribution prohibited without permission

How Far Forward Each Method Moves the Distribution

	what moves Γ	horizon in the loss	gradient through the move?
DeepHAM	a panel of $N = 50$, simulated	$T_{\text{unroll}} = 150$ periods	yes, through the sampled panel
Deep Equilibrium Nets	Young's lottery on a histogram	one period	no: successors are stored data
Sequence space	Young's lottery, beside the history	one period	no: prices and K' held fixed
Feng, Han, Sargent & Zhu (2026)	the lottery matrix $\widehat{\Phi}(\mathbf{x};\theta)$, in the graph	T: 100; 30 with a government	yes, through every $\widehat{\Phi}_t$

- **Methods 2 and 3 use the one-period lottery only to generate data; Feng, Han, Sargent & Zhu (2026) differentiate the whole path, DeepHAM through a sample.**

Source: Feng, Han, Sargent & Zhu (September 2026), preliminary; the three sources' code, as on the previous pages.

© Zhigang Feng. Redistribution prohibited without permission

Argument 1 (a): What a Markov Government Has to Compute

- Each period the government picks a tax τ; households respond with g_τ; successors follow the rule $\tau(\cdot)$:

$$V^G(\Gamma, z) = \max_{\tau} \Big\{ W(\Gamma, z, \tau) + \beta\, \mathbb{E}\big[V^G(\Gamma', z') \mid z \big] \Big\}, \qquad \Gamma' = H\big(\Gamma, z, z';\, g_\tau\big),$$

  and in equilibrium the maximiser is the rule itself: $\tau(\Gamma, z)$.
- Its first-order condition:

$$\frac{\partial W}{\partial \tau} + \beta\, \mathbb{E}\Big[\frac{\partial V^G}{\partial \Gamma'} \cdot \frac{\partial H}{\partial g} \cdot \frac{\partial g_\tau}{\partial \tau} \Big] = 0.$$

- H is the law of motion, not a histogram: it maps today's distribution to tomorrow's. The second term: τ changes saving g_τ, which moves Γ' through H; with $\beta R \approx 1$ it compounds.
- **Needed: a rule that reads Γ, households that respond to τ, and $d\Gamma'/d\tau$ through H.**

Source: Feng, Han, Sargent & Zhu (September 2026), preliminary; Klein, Krusell & Ríos-Rull, *RES* (2008).

© Zhigang Feng
Redistribution prohibited without permission

Argument 1 (b): Which Method Supplies What

	rule reads Γ	**households respond to τ**	$d\Gamma'/d\tau$	**successor governments**
DeepHAM	through a moment of 50	tax fixed; a planner option picks *c*	through a sampled panel	no
Deep Eq. Nets	yes, the histogram	not in the code	no: Γ' is stored data	no
Sequence space	no: shocks only	not in the code	no: prices held fixed	no
Feng, Han, Sargent & Zhu (2026)	yes, the histogram	yes: implementability	yes: through $\widehat{\Phi}$, every date	yes: a perceived rule

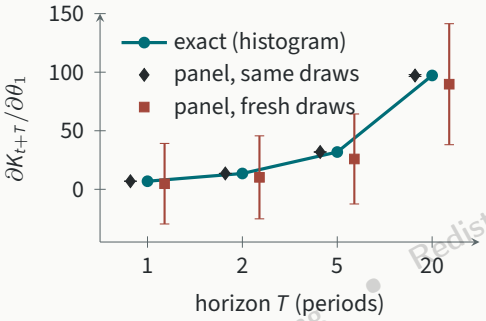
Feng, Han, Sargent & Zhu (2026) so far without aggregate risk: adding *z* is open.

- The sequence space fails at the root: history $\rightarrow \Gamma$ holds under *one fixed* policy; a deviating government breaks it. **Only a law of motion in the graph supplies the second term of the government's condition.**

Source: Feng, Han, Sargent & Zhu (September 2026), preliminary; Han, Yang & E, arXiv 2112.14377, §2.4, §5; the three sources' code.

© Yaohua Feng Redistribution prohibited without permission

Argument 2 (a): A Derivative From a Panel, and From the Histogram

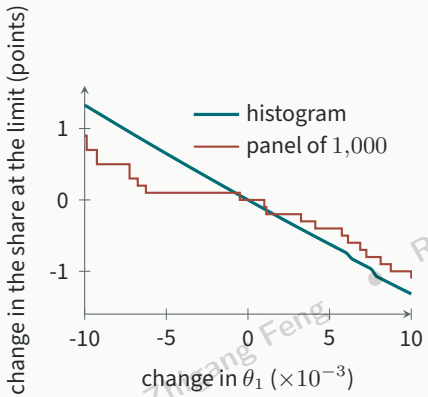


- **The test**: households save $a' = \theta_0 + \theta_1$ cash, clipped at the limit. How far does capital T periods on move per unit of θ_1?
- **Exact**, through the histogram: 6.90 at $T = 1$.
- **A panel** of $10{,}000$: simulate at $\theta_1 \pm 0.001$ and difference. Fresh draws: 4.8 ± 34.4; same draws: 6.90 ± 0.05. Mean $\pm$ one s.d. over 20 repetitions, as the bars.
- **For capital a panel works if its draws are fixed, as in DeepHAM's gradient.**

© Zhigang Feng · Redistribution prohibited without permission

Source: Lab block B: $\theta_1 = 0.938$, a linear rule fitted to the solved policy; the lab's annual economy.

Argument 2 (b): Where a Panel Breaks: Head Counts and Clamps

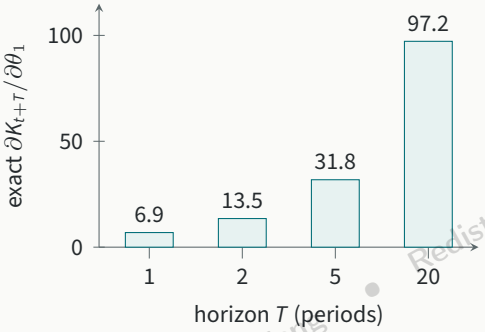


- Same test, now the share at the borrowing limit after 20 periods; one household is 0.1 point.
- A step's derivative is zero wherever it exists. At $T = 2$: exact -0.13; panel 0.00 ± 0.00.
- A clamped household ignores θ_1: a panel gradient misses who is pushed onto the limit.
- **Only the lottery in the graph, moving mass $h\,da'/\Delta$, gives the whole derivative.**

© Zhiyang Feng Redistribution prohibited without permission

Source: Lab block B's test swept over θ_1, same draws at every slope; the lab's annual economy. Feng, Han, Sargent & Zhu (September 2026), preliminary.

Argument 2 (c): Why the Derivative Must Look Many Periods Ahead



- At $T = 20$ the effect is 14 times its one-period size, and still rising; five periods capture 33% of it.
- With $\beta R \approx 1$ an effect lives for about $1/(1-\beta R)$ periods: about 420 in Deck 4.1's economy.
- A one-period condition (Methods 2 and 3) sees only the first bar.
- **DeepHAM sees $T_{\text{unroll}} = 150$ periods, through a sample; Feng, Han, Sargent & Zhu (2026), 100 or 30, exactly.**

© Zhigang Feng · Redistribution prohibited without permission

Source: Lab block B, the exact histogram derivative; the lab's annual economy. Feng, Han, Sargent & Zhu (September 2026), preliminary.

Bridge: Each Tool for Its Question

Sequence space: long memory, stable training, an exogenous domain.

Deep Equilibrium Nets: the whole histogram as state; first-order conditions.

DeepHAM: interpretable moments; long rollouts.

Feng, Han, Sargent & Zhu (2026): the law of motion in the graph; a government without commitment.

- For the competitive equilibrium of this economy, all three work, and the classical rule is already accurate.
- The difference opens when someone chooses a policy that moves the distribution, and must be consistent with later choices.
- **Deck 4.3: households that age, and aggregate risk over cohorts, where the third method earns its place.**

© Zhigang Feng Redistribution prohibited without permission

Source: Feng, Han, Sargent & Zhu (September 2026), preliminary; the sources of this deck.